

데이터를 분석하려면 무엇이 필요할까?

데이터는 어디에 저장되고 어떻게 읽을까?



Excel



CSV



Database



The background of the slide features a person sitting at a desk, viewed from behind, looking at a laptop and several large monitors. The monitors display various data visualizations: a world map with glowing nodes, a bar chart, a line graph, and a donut chart. The laptop screen shows a table of data.

ID	Name	Age	Score	Major
001	Kim	21	90	AI
002	Lee	20	85	Data
003	Park	22	92	AI
004	Choi	21	88	Data
005	Choi	21	78	AI
005	Jung	20	78	AI
...	...	...	...	...



Python을 활용한 데이터분석
2강 : Pandas와 DataFrame 이해하기

그런데 진짜 데이터는 어디에 있을까?

우리가 사용하는 서비스와 기업, 기관은
매일 엄청난 양의 데이터를 만들어 내고
어딘가에 저장하고 있습니다.



생성

매일 새로운 데이터가
계속 생성됩니다.



저장

파일, 데이터베이스 등
다양한 형태로 저장됩니다.



활용

분석과 의사결정을 위해
데이터를 활용합니다.

지난 시간 우리는 무엇을 배웠을까?

1강 : 데이터와 파이썬 기초

01



AI와 데이터

AI는 데이터를 학습하여
패턴을 찾고 예측한다



02



데이터 분석 개요

데이터를 수집, 정리, 분석하여
의미있는 정보를 얻는다



03



Python 기초

Python은 데이터 분석에
많이 사용되는 언어이다



04



기본 문법 학습

변수, 리스트, 반복문,
조건문, 함수를 학습했다



05



문제 해결의 시작

기본 문법을 통해
문제를 해결할 수 있다



이제 우리는
데이터를 다룰 준비가 되었습니다!



이번 시간부터는
실제 데이터를 읽고 분석하는 방법을
배워보겠습니다!



데이터는 다양한 형태로 존재한다

데이터는 다양한 형식으로 저장되며, 분석을 위해서는 먼저 데이터를 읽을 수 있어야 합니다.



Excel

표 형태로 정리된 데이터
가장 많이 사용하는 데이터 형식

	A	B	C	D
1	이름	국어	수학	총점
2	김민수	85	90	175
3	이지현	92	88	180
4	박준호	78	85	163
5	최수빈	88	94	182
6	정다은	91	87	178
7	...	...	...	...



CSV

쉼표로 구분된 텍스트 파일
가볍고, 호환성이 높아 분석에 최적

```
name,korean,math,total
Kim,85,90,175
Lee,92,88,180
Park,78,85,163
Choi,88,94,182
Jung,91,87,178
...
```



JSON

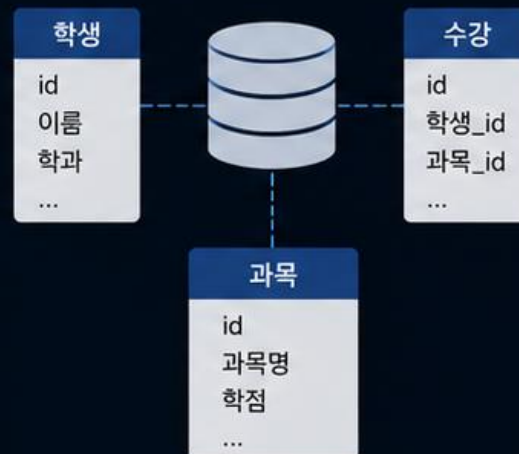
계층 구조(키-값)로 저장
웹 서비스, 앱에서 많이 사용

```
{
  "students": [
    {"name": "Kim", "korean": 85,
      "math": 90, "total": 175},
    {"name": "Lee", "korean": 92,
      "math": 88, "total": 180},
    {"name": "Park", "korean": 78,
      "math": 85, "total": 163}
  ]
}
```



Database

데이터를 체계적으로 저장
대용량 데이터 관리에 적합



다양한 형태의 데이터를 다루기 위해,
우리는 데이터를 읽고 이해하는 방법을 배워야 합니다!



데이터 파일



표(Table)로 이해



Pandas DataFrame

우리가 가장 많이 보는 데이터, Excel

학교 성적표, 회사 업무 보고서, 매출 관리표 등
우리 주변의 많은 데이터가 **Excel 파일로 저장**됩니다.



익숙한 표 형태

행과 열로 구성되어 있어
데이터를 한눈에 확인하기 쉽습니다.



다양한 기능

계산, 정렬, 필터링, 차트 등
다양한 기능을 제공합니다.



널리 사용

업무, 교육, 행정 등
여러 분야에서 널리 활용됩니다.

성적표.xlsx

파일 홈 삽입 페이지 레이아웃 수식 데이터 검토 보기 도움말

붙여넣기 글꼴 맞춤 표시 형식 조건부 서식 표 서식 셀 스타일 삽입 삭제 서식 정렬 및 필터 찾기 및 선택

A1 이름

	A	B	C	D	E	F	G
1	이름	학과	중간고사	기말고사	과제	총점	등급
2	김민수	경영학과	85	90	95	270	A
3	이지현	컴퓨터공학과	92	88	90	270	A
4	박준호	경제학과	78	85	88	251	B+
5	최수빈	통계학과	88	92	93	273	A
6	정다은	경영학과	91	87	90	268	A-
7	이서연	컴퓨터공학과	82	80	85	247	B
8	한지우	경제학과	76	82	80	238	B
9	...	...	...	...	...	...	...



Excel은 데이터를 관리하고 확인하는 데 매우 유용하지만,
데이터 분석을 자동으로 처리하기에는 한계가 있습니다.



더 많은 데이터를 빠르고 효율적으로
분석하려면 어떻게 해야 할까요?

Excel만으로 충분할까?

Excel은 편리하지만, 데이터가 많아지고
반복적인 작업이 필요해지면 **한계**가 있습니다.

- Excel의 한계



데이터가 많아지면 느려진다

수만 ~ 수십만 행의 데이터는 파일 크기가 커지고,
열고 저장하는 데 시간이 오래 걸립니다.



매일 반복되는 작업이 비효율적이다

새 데이터가 생길 때마다 복사, 붙여넣기, 정리해야 합니다.
실수할 가능성도 높아집니다.



협업과 공유가 불편하다

여러 사람이 동시에 작업하기 어렵고,
파일 버전 관리가 복잡해집니다.



데이터가 많아지고 분석을 자동화하려면,
더 **효율적인 방법**이 필요합니다!



그래서 우리는 '**CSV**'라는
더 가볍고 효율적인 데이터를 사용합니다.

이런 경험, 한 번쯤 있으시죠?



파일 열기/저장 시간이 오래 걸림



시간이 오래 걸려
기다리는 일이 많아집니다.



데이터 복사 → 붙여넣기 → 정리



반복 작업이 많아
실수가 발생하기 쉽습니다.



파일 공유 및 버전 관리



어떤 작업이 최신 버전인지
헷갈리기 쉽습니다.

그래서 CSV를 사용한다!

Excel 파일을 그대로 분석하기는 어렵습니다.

그래서 데이터를 더 가볍고 다루기 쉬운 'CSV' 파일로 변환하여 사용합니다.

CSV란?



Comma Separated Values의 약자로,
데이터를 '쉼표(,)'로 구분하여 저장하는
간단한 텍스트 파일 형식입니다.

CSV의 장점



가볍다

용량이 작아
저장과 전송이 쉽다



빠르다

읽고 쓰는 속도가
매우 빠르다



호환성이 높다

다양한 프로그램과
언어에서 사용 가능



분석에 최적화

데이터 분석 도구에서
쉽게 처리할 수 있다

Excel에서 CSV로 변환하는 과정

Excel 파일 (예: 성적표.xlsx)



이름	국어	수학	영어	총점
김민수	85	90	88	263
이지현	92	88	90	270
박준호	78	85	88	251
최수빈	88	92	93	273
정다운	91	87	90	268
...				...



다른 이름으로 저장
"CSV (쉼표로 분리)"

CSV 파일 (예: 성적표.csv)



Excel은 사람이 보기 편한 형식,
CSV는 컴퓨터가 읽고 분석하기 편한 형식입니다!



이제 우리는 CSV 파일을 통해
데이터를 파이썬에서 읽고 분석할 수 있습니다!

CSV 파일, 열어보기!

CSV 파일은 텍스트 파일입니다.

엑셀과 달리 표처럼 보이지 않지만,

쉼표(,)로 구분된 데이터를 가지고 있습니다.



CSV 파일 특징

- ✓ 사람이 읽을 수 있는 텍스트 파일
- ✓ 각 값은 쉼표(,)로 구분
- ✓ 어떤 프로그램에서도 열고 저장 가능

scores.csv (메모장으로 열었을 때)

```

1  이름 , 학과 , 중간고사 , 기말고사 , 과제 , 총점
2  김민수 , 경영학과 , 85 , 90 , 95 , 270
3  이지현 , 컴퓨터공학과 , 92 , 88 , 90 , 270
4  박준호 , 경제학과 , 78 , 85 , 88 , 251
5  최수빈 , 통계학과 , 88 , 92 , 93 , 273
6  정다운 , 경영학과 , 91 , 87 , 90 , 268
7  ...
8
9
  
```

1행(헤더)

각 열(컬럼)의 이름

2행 이후

각 행(레코드)의 데이터

쉼표(,)

각 값(필드)을 구분



지금 보이는 이 파일이 바로
우리가 분석할 **데이터의 원본**입니다!



CSV 파일



표(Table)



Pandas DataFrame

CSV는 왜 중요할까?

CSV는 데이터를 저장하고 공유하는 가장 널리 사용되는 **표준 형식**입니다.



가볍다

파일 크기가 작아 저장과 전송이 쉽습니다.



빠르다

읽고 쓰는 속도가 빨라 대용량 데이터에 유리합니다.



호환성이 높다

엑셀, 파이썬, R, SQL 등 대부분의 도구에서 쉽게 사용할 수 있습니다.



공유와 활용이 쉽다

다른 사람과 데이터를 주고받거나 웹 / 시스템에 업로드하기 좋습니다.



CSV는 **단순하지만 강력한 형식**입니다.
그래서 데이터 분석을 시작할 때 가장 먼저 배우고 사용하는 형식입니다!

CSV는 다양한 환경에서 사용됩니다



Excel



Python
(Pandas)



R



데이터베이스
(SQL)



BI 도구
(Tableau 등)



웹/클라우드
서비스

CSV vs 다른 데이터 형식

형식	사람이 읽기 쉬운가?	파일 크기	처리 속도	호환성	용도
CSV	매우 쉬움	작음	빠름	매우 높음	데이터 분석, 교환, 저장
Excel (.xlsx)	쉬움	큼	보통	높음	수작업 데이터 관리
JSON	보통	보통	보통	보통	웹/프로그램 간 데이터 교환
Database	어려움	상대적으로 큼	빠름	높음	대규모 운영 데이터 관리



데이터 저장



데이터 읽기



데이터 분석

데이터는 결국 표(Table)다!

우리가 다루는 대부분의 데이터는
행(Row)과 열(Column)로 구성된 표 형태입니다.

✓ 표(Table)의 특징

- 데이터를 체계적으로 정리할 수 있다.
- 비교, 검색, 계산이 쉽다.
- 컴퓨터가 이해하고 처리하기 가장 좋은 형태이다.

다양한 데이터도 결국은 표로 표현할 수 있습니다

설문 조사 결과



ID	성별	만족도
1	남	4
2	여	5
3	남	3
...		

매출 데이터



날짜	상품	매출
4/1	A	120,000
4/1	B	80,000
4/2	A	150,000
...		

로그 데이터



시간	사용자	행동
10:01	user1	login
10:02	user2	click
10:03	user1	view
...		

센서 데이터



시간	온도(°C)	습도(%)
10:00	23.4	45
10:01	23.6	44
10:02	23.5	46
...		

표(Table)의 구조

표(Table)의 구조

열 (Column)
같은 종류의 데이터

이름	국어	수학	영어	총점
김민수	85	90	88	263
이지현	92	88	90	270
박준호	78	85	88	251
...				

행 (Row)
데이터 한 줄
(하나의 관측)



행과 열로 이루어진 표 구조를
우리가 '데이터의 기본 형태'로
이해하는 것이 중요합니다!



이제 우리는 CSV 파일을 통해 표 형태의 데이터를 다루었습니다.
다음 단계에서는 이 표 데이터를 Python에서 어떻게 다룰 수 있는지 알아보시다!

행(Row)이란?

표(Table)에서 가로로 나열된 한 줄을 **행(Row)**이라고 합니다.



핵심 포인트

- 한 행은 하나의 '관측 단위'를 의미합니다.
- 예) 한 학생의 성적, 한 번의 주문, 한 번의 로그인
- 행이 많아질수록 데이터의 양이 많아집니다.

예시: 학생 성적 데이터

	이름	국어	수학	영어	총점
1	김민수	85	90	88	263
2	이지현	92	88	90	270
3	박준호	78	85	88	251
4	최수빈	88	92	93	273
...	...	...	...	...	...

1번 행 = 김민수의 성적

한 행은
하나의 개체(관측 단위)에 대한
모든 정보를 담고 있습니다.

일상 속 비유로 이해하기



- 1번째 책
- 2번째 책
- 3번째 책
- ...

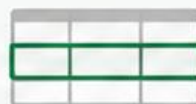
책 한 권이
하나의 '정보 단위'이듯,
표의 **한 행**은 하나의
'**데이터 단위**'입니다.

정리



표(Table)

데이터를 표 형태로
정리한 것



행(Row)

표에서 가로로 나열된
한 줄



하나의 관측 단위

한 행은 하나의 대상에 대한
모든 정보를 포함



행이 많아질수록

더 많은 데이터가
축적됩니다.

데이터는 다양한 형태로 존재한다

우리가 다루는 데이터는 표 형태뿐만 아니라
텍스트, 숫자, 이미지, 시간, 위치 등 **다양한 형태**로 존재합니다.



기억하세요!

모든 데이터는 형태가 다르지만,
컴퓨터는 0과 1로 표현하여 저장하고 처리합니다.

1. 표 (구조화 데이터)

행(Row)과 열(Column)이
있는 표 형태

이름	수학	영어	총점
김민수	85	90	175
이지현	92	88	180
...	...	...	...

예: 성적표, 매출 데이터

2. 텍스트 데이터

문자, 문장 등
자연어 형태

"오늘 날씨가 좋아서
기분이 정말 좋았어요."



예: 리뷰, 기사, 이메일

3. 숫자 데이터

정수, 실수 등
숫자 형태

2024, 100, -5
3.14, -0.25, 98.6

예: 온도, 가격, 개수

4. 이미지 데이터

사진, 그림 등
시각적 정보



예: 상품 이미지, 사진

5. 시간 데이터

날짜와 시간 정보



2024-05-21
15:30:25

예: 로그인 시간, 주문 시간

6. 위치 데이터

위도, 경도 등
공간 정보



위도: 37.5665
경도: 126.9780

예: 매장 위치, 배달 위치



핵심 정리

데이터는 다양한 형태로 존재하지만,
분석을 위해서는 먼저 **데이터를 읽고 이해할 수 있어야** 합니다.



다음 단계에서는
우리가 가장 많이 다루는 **표(구조화 데이터)**부터
Python으로 읽는 방법을 배워봅시다!

컴퓨터는 어떻게 데이터를 이해할까?

컴퓨터는 모든 데이터를 0과 1로 표현합니다.

CSV 파일의 표 데이터도 컴퓨터 메모리에서는
데이터 타입에 따라 다르게 저장되고 처리됩니다.

CSV 파일



데이터프레임(DataFrame)

이름	국어	수학	영어	총점
김민수	85	90	88	263
이지현	92	88	90	270
박준호	78	85	88	251
...	...	...	...	...

각 열(컬럼)은 하나의 데이터 타입을 가집니다.



핵심 포인트

- 모든 데이터는 메모리 안에 저장됩니다.
- 각 열(컬럼)은 하나의 데이터 타입을 가집니다.
(숫자, 텍스트, 날짜, 참/거짓 등)
- 올바른 데이터 타입은 분석의 정확도와 효율성에 큰 영향을 줍니다.

주요 데이터 타입

123

정수형 (int)

소수점이 없는 숫자

예) 85, 90, 263

저장 방식

이진수로 저장 1010101
(메모리 사용 효율적) (이진수)

1.5

실수형 (float)

소수점이 있는 숫자

예) 98.6, -0.25, 3.14

저장 방식

부동소수점 방식 3.14
(더 넓은 범위 표현 가능) (이진수)

A

문자열 (string)

텍스트 데이터

예) 김민수, 서울, "좋아요"

저장 방식

문자 인코딩 김 = EAB0 80
(예: UTF-8) (UTF-8 예시)

📅

날짜/시간 (datetime)

날짜와 시간 정보

예) 2024-05-21 15:30:25

저장 방식

시간을 숫자로 변환 1716283825
(예: 타임스탬프) (타임스탬프)

✓

논리형 (bool)

참(True) 또는 거짓(False)

예) True, False

저장 방식

1 (True) 또는 0 (False) 1 / 0
로 저장



데이터 처리 과정

데이터는 올바른 타입으로
저장되고 처리되어야
정확한 분석이 가능합니다.

1

CSV 파일 읽기



파일의 텍스트 정보를
읽어옵니다.

2

데이터 타입 추론



각 열의 데이터를 보고
타입을 추정합니다.

3

타입 변환



필요에 따라 올바른
데이터 타입으로 변환합니다.

4

메모리에 저장



0과 1(이진수)로 변환되어
메모리에 저장됩니다.

5

연산 및 분석



저장된 데이터를 이용해
계산하고 분석합니다.

Python은 데이터를 어떻게 읽을까?

파이썬은 다양한 파일 형태의 데이터를 읽을 수 있습니다.
그중에서 가장 많이 사용하는 방법은 **CSV 파일**을 읽어 **DataFrame**으로 변환하는 것입니다.

CSV 파일 읽기의 3단계



Python이 읽을 수 있는 주요 파일 형식



Python 코드 예시

```
1 import pandas as pd
2
3 # CSV 파일 읽기
4 df = pd.read_csv('data.csv')
5
6 # 처음 몇 행 확인
7 print(df.head())
```

실행 결과 (일부분)

	이름	국어	수학	영어	총점
0	김민수	85	90	88	263
1	이지현	92	88	90	270
2	박준호	78	85	88	251
3	최수빈	88	92	93	273
...	...	...	...	...	...



핵심 포인트

- pandas의 **read_csv()** 함수로 CSV 파일을 쉽게 읽을 수 있습니다.
- 읽은 데이터는 **DataFrame**이라는 표 형태의 자료구조로 변환됩니다.
- DataFrame은 분석과 처리에 매우 편리합니다.



지금까지의 흐름을 정리해봅시다!



다음 단계

다음 슬라이드에서는 **Pandas** 라이브러리의 역할과 장점에 대해 알아보십시오!

Pandas의 등장

Python에서 데이터를 분석하고 처리하기 위해 가장 널리 사용되는 라이브러리가 바로 “Pandas”입니다. Pandas는 데이터를 쉽게 읽고, 정리하고, 분석할 수 있도록 도와주는 강력한 도구입니다.



Pandas란 무엇인가?

- ✓ Python으로 작성된 오픈소스 라이브러리
- ✓ 데이터 분석과 조작을 위한 고수준의 도구 제공
- ✓ 2008년 Wes McKinney에 의해 개발
- ✓ NumPy를 기반으로 동작
- ✓ DataFrame이라는 효율적인 자료구조 제공



“ Pandas는 데이터를 다루기 위한 ‘편리하고 강력한 도구 상자’입니다. 복잡한 코딩 없이 데이터 분석에 집중할 수 있게 해줍니다.

Pandas가 제공하는 주요 기능



데이터 읽기/쓰기

CSV, Excel, SQL 등 다양한 형식의 데이터를 쉽게 읽고 저장



데이터 정리/변환

결측치 처리, 데이터 필터링, 정렬, 그룹화 등 데이터 전처리 기능 제공



데이터 분석

통계 요약, 집계, 피벗 테이블, 시계열 분석 등 다양한 분석 도구 제공



데이터 시각화 연동

Matplotlib, Seaborn 등 시각화 라이브러리와 손쉽게 연동



성능과 편의성

대용량 데이터도 빠르게 처리, 직관적인 문법으로 생산성 향상



Pandas의 역할

Pandas는 CSV 파일과 같은 다양한 데이터를 Python의 **DataFrame**이라는 표 형태로 변환하여, 데이터 분석의 모든 과정을 쉽게 만들어줍니다.



데이터 가져오기
(읽기)



DataFrame으로 변환
(정리/가공)



분석하기
(탐색/계산)



결과 활용
(시각화/의사결정)



핵심 한 줄 요약

Pandas는 데이터를 이해하고 분석하기 쉽게 만들어주는 Python의 핵심 도구!

DataFrame이란?

Pandas에서 가장 기본이 되는 데이터 구조입니다.

행(Row)과 **열(Column)**을 가지는 **2차원 표** 형태의 데이터로, Excel의 시트와 유사하지만 더 강력하고 유연합니다.



핵심 포인트

- DataFrame은 표(Table)와 같은 구조를 가집니다.
- 각 열은 서로 다른 데이터 타입을 가질 수 있습니다.
- 행과 열에 이름(인덱스, 컬럼명)을 가질 수 있습니다.

DataFrame 구조 이해하기

열(Column)						
	이름	국어	수학	영어	총점	
행 (Row)	0	김민수	85	90	88	263
	1	이지현	92	88	90	270
	2	박준호	78	85	88	251
	3	최수빈	88	92	93	273

1개의 행(Row)
한 학생의 모든 정보

1개의 열(Column)
모든 학생의 수학 점수

- 인덱스(Index) : 행의 이름 (0,1,2,...)
- 컬럼(Coluns) : 열의 이름

DataFrame의 특징



2차원 구조

행과 열로 이루어진 표 형태의 데이터



다양한 데이터 타입

열마다 서로 다른 데이터 타입 가능



라벨 기반 접근

행/열 이름으로 쉽게 조회 가능



강력한 연산 기능

필터링, 정렬, 그룹화, 통계 등 다양한 기능 지원



다양한 데이터 소스 지원

CSV, Excel, DB, API 등 여러 소스에서 로드 가능

DataFrame 생성 예시

Python 코드

```
1 import pandas as pd
2
3 data = {
4     '이름': ['김민수', '이지현', '박준호', '최수빈'],
5     '국어': [85, 92, 78, 88],
6     '수학': [90, 88, 85, 92],
7     '영어': [88, 90, 88, 93]
8 }
9 df = pd.DataFrame(data)
10 print(df)
```

실행 결과(DataFrame)

	이름	국어	수학	영어
0	김민수	85	90	88
1	이지현	92	88	90
2	박준호	78	85	88
3	최수빈	88	92	93

<class 'pandas.core.frame.DataFrame'>



한 줄 요약

DataFrame은 데이터를 분석하기 위한 가장 기본이 되는 '표 형태의 데이터 구조'입니다.

DataFrame의 기본 구조와 접근

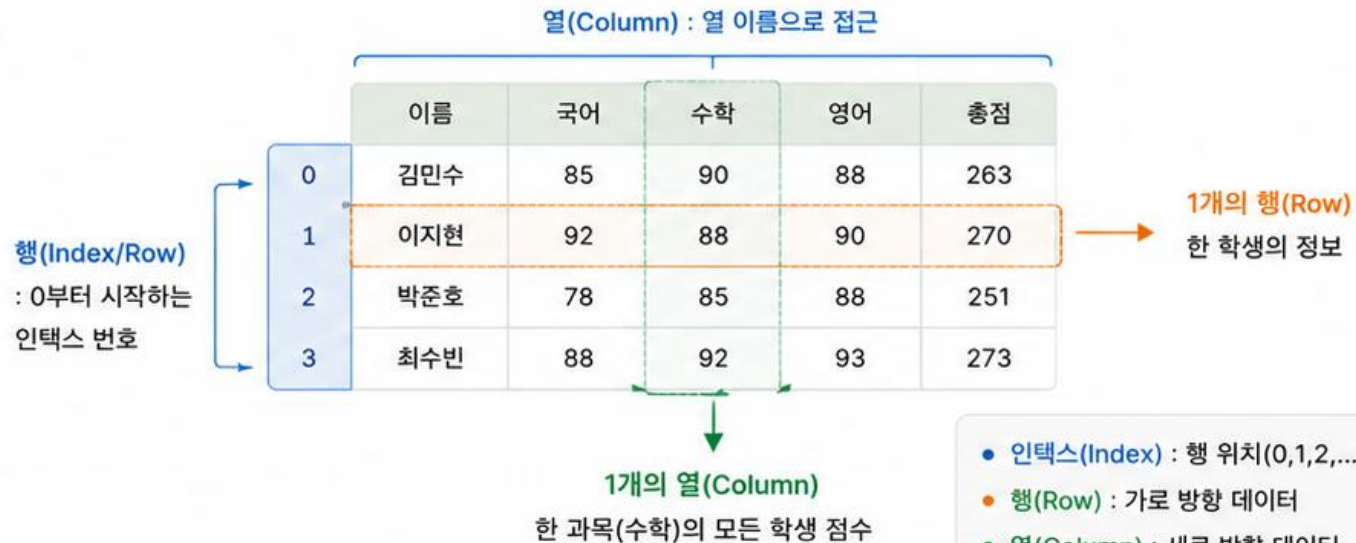
DataFrame은 행(Row)과 열(Column)로 이루어진 2차원 표입니다. 우리는 행과 열을 선택하거나, 특정 값을 조회하고, 필터를 적용할 수 있습니다.



핵심 포인트

- df : 전체 DataFrame
- df['열이름'] : 특정 열(Series) 선택
- df.loc[행, 열] : 라벨 기반 접근
- df.iloc[행, 열] : 위치(숫자) 기반 접근
- 슬라이싱과 필터링으로 원하는 데이터를 추출할 수 있습니다.

DataFrame 구조 다시 보기



1 특정 열 선택

df['수학']

	수학
0	90
1	88
2	85
3	92

결과 타입: Series

2 여러 열 선택

df[['이름', '총점']]

	이름	총점
0	김민수	263
1	이지현	270
2	박준호	251
3	최수빈	273

결과 타입: DataFrame

3 행 선택 (라벨 기반)

df.loc[1] 또는 df.loc[[0,2]]

	이름	국어	수학	영어	총점
1	이지현	92	88	90	270
0	김민수	85	90	88	263
2	박준호	78	85	88	251

결과 타입: Series 또는 DataFrame

4 행 선택 (위치 기반)

df.iloc[1] 또는 df.iloc[0:2]

	이름	국어	수학	영어	총점
1	이지현	92	88	90	270
0	김민수	85	90	88	263

결과 타입: Series 또는 DataFrame

조건에 맞는 데이터 필터링 예시

```
# 수학 점수가 90점 이상인 학생
df[df['수학'] >= 90]

# 총점이 260점 이상인 학생
df[df['총점'] >= 260]

# 국어 점수가 80점 이상이고, 영어 점수가 90점 이상인 학생
df[(df['국어'] >= 80) & (df['영어'] >= 90)]
```

	이름	국어	영어	총점
1	이지현	92	88	970
3	최수빈	88	92	273



Tip

조건식은 True/False로 계산되며, True인 행만 결과로 반환됩니다. (불리언 인덱싱)



한 줄 요약

DataFrame은 표 형태의 데이터를 다루기 위한 가장 강력한 도구이며, 다양한 방법으로 원하는 데이터를 선택하고 추출할 수 있습니다.

DataFrame으로 무엇을 할 수 있을까?

DataFrame은 단순히 데이터를 출력하는 것을 넘어,
선택, 정렬, 필터링, 요약, 그룹화 등
다양한 데이터 분석 기능을 제공합니다.

DataFrame의 주요 기능



데이터 조회

특정 행/열을 선택하고
조건에 맞는 데이터를
쉽게 조회할 수 있습니다.



정렬

특정 열(컬럼)을 기준으로
오름차순 또는 내림차순
정렬이 가능합니다.



그룹화(GroupBy)

특정 기준으로 그룹을 나누고
그룹별로 요약/집계할 수
있습니다.



데이터 필터링

조건을 지정하여 원하는
데이터만 추출할 수
있습니다.



요약 통계

합계, 평균, 최댓값, 최솟값,
개수 등 다양한 통계 정보를
계산할 수 있습니다.



결과 저장

분석한 결과를 CSV, Excel,
데이터베이스 등 다양한
형식으로 저장할 수 있습니다.

예제 데이터 (학생 성적)

	이름	국어	수학	영어	학년	반	성별
0	김민수	85	90	88	2	1	남
1	이지현	92	88	90	2	1	여
2	박준호	78	85	88	2	2	남
3	최수빈	88	92	93	2	2	여
4	정우진	90	80	85	3	1	남



DataFrame을 활용하면
이 데이터를 다양한 방법으로
분석하고 활용할 수 있습니다!

Python 코드 예시

```

1 import pandas as pd
2
3 df = pd.read_csv('student.csv') # CSV 파일 읽기
4
5 # 1) 특정 열 선택
6 print(df['이름'])
7
8 # 2) 수학 점수가 90점 이상인 학생 필터링
9 high_math = df[df['수학'] >= 90]
10 print(high_math)

```

수행 결과 예시

1 특정 열 선택 (이름)

```

0 김민수
1 이지현
2 박준호
3 최수빈
4 정우진

```

Name: 이름, dtype: object

2 수학 점수가 90점 이상인 학생

	이름	국어	수학	영어	학년	반	성별
0	김민수	85	90	88	2	1	남
3	최수빈	88	92	93	2	2	여



핵심 포인트

DataFrame은 데이터 분석의 시작점이자 중심 도구입니다.
데이터를 자유롭게 탐색하고, 원하는 정보를 빠르게 얻을 수 있도록 도와줍니다.



한 줄 요약

DataFrame은 데이터를 '보기 쉽게' 만들고, '분석하기 쉽게' 만들어주는 강력한 도구입니다.

DataFrame

기본 조작하기

DataFrame은 다양한 방법으로 데이터를 선택하고, 새로운 정보를 만들고, 필요에 맞게 변형할 수 있습니다. 가장 많이 사용하는 기본 조작 방법을 알아봅시다.



핵심 포인트

- DataFrame은 표 형태 데이터를 다루기 위한 강력한 구조입니다.
- 원하는 행, 열, 조건에 맞는 데이터를 쉽게 추출할 수 있습니다.
- 새로운 열을 만들거나 데이터를 정렬하는 것도 간단합니다.

2. 조건에 맞는 데이터 추출하기

(1) 조건식으로 필터링

조건을 만족하는 행만 추출합니다.

```
df[df['수학'] >= 90]
# 수학 점수가 90점 이상인 학생
```

출력 결과 (DataFrame)

	이름	국어	수학	영어	학년	반	성별
0	김민수	85	90	88	2	1	남
3	최수빈	88	92	93	2	2	여

(2) 여러 조건 결합

여러 조건을 만족하는 데이터를 추출합니다.

```
df[(df['국어'] >= 80) & (df['수학'] >= 90)]
# 국어 80점 이상이고 수학 90점 이상
```

출력 결과 (DataFrame)

	이름	국어	수학	영어	학년	반	성별
0	김민수	85	90	88	2	1	남
3	최수빈	88	92	93	2	2	여



자주 사용하는 속성/메서드

속성/메서드	설명	속성/메서드	설명
df.head(n)	처음 n개 행 확인 (기본 5개)	df.info()	데이터 요약 정보 확인
df.tail(n)	마지막 n개 행 확인 (기본 5개)	df.describe()	수치형 데이터 통계 요약
df.shape	행과 열의 개수 확인	df.columns	열 이름 확인



핵심 정리

- ✓ DataFrame은 다양한 방법으로 데이터를 선택하고 변형할 수 있습니다.
- ✓ 필터링, 정렬, 새로운 열 생성 등으로 원하는 형태의 데이터를 만들 수 있습니다.
- ✓ 이 기능들을 활용하면 데이터 분석이 훨씬 쉬워집니다!

1. 데이터 선택하기

(1) 열(컬럼) 선택

하나의 열(Series)을 선택합니다.

```
df['수학']
# '수학' 열 선택
```

출력 결과 (Series)

	0	90
dtype:	int64	
1	88	
2	85	
3	92	
4	80	

(2) 여러 열(컬럼) 선택

여러 개의 열을 선택합니다.

```
df[['국어', '영어']]
# '국어', '영어' 열 선택
```

출력 결과 (DataFrame)

	국어	영어
0	85	88
1	92	90
2	78	88
3	88	93
4	90	85

(3) 특정 행(인덱스) 선택

특정 행의 데이터를 선택합니다.

```
df.loc[1]
# 인덱스 1번째 행 선택
```

출력 결과 (Series)

	이름	이지현
dtype:	object	
국어	92	
수학	88	
영어	90	
학년	2	
반	1	
성별	여	
Name:	1,	dtype: object

3. 데이터 변형하기

(1) 새 열(컬럼) 만들기

기존 데이터를 이용해 새로운 열을 만들 수 있습니다.

```
df['총점'] = df[['국어', '수학', '영어']].sum(axis=1)
# 국어, 수학, 영어 합계
```

출력 결과 (DataFrame)

	이름	국어	영어	총점
0	김민수	85	98	263
1	이지현	92	90	270
2	박준호	78	88	251
3	최수빈	88	92	273
4	정우진	90	85	255

(2) 정렬하기

특정 열을 기준으로 데이터를 정렬합니다.

```
df.sort_values('총점', ascending=False)
# 총점 기준 내림차순 정렬
```

출력 결과 (DataFrame)

	이름	국어	수학	총점
3	최수빈	88	93	273
1	이지현	92	90	270
0	김민수	85	88	263
4	정우진	90	85	255
2	박준호	78	88	251

TIP

DataFrame은 불변(immutability)이 아니라 가변(mutable) 구조이므로, 변경된 내용은 원본 DataFrame에 반영됩니다. 필요 시 `df.copy()`로 복사해 사용하세요!

DataFrame

활용 예시 모음

DataFrame은 다양한 메서드와 속성을 제공하여 데이터를 쉽고 빠르게 탐색하고 분석할 수 있습니다. 가장 자주 사용하는 기능들을 함께 살펴봅시다.



핵심 포인트

- DataFrame은 다양한 방법으로 데이터를 조회, 필터링, 정렬, 요약할 수 있습니다.
- 간결한 코드로 복잡한 분석 작업을 수행할 수 있습니다.
- 메서드 조합을 통해 원하는 정보를 유연하게 얻을 수 있습니다.

Python 코드 (데이터 불러오기)

```
import pandas as pd

df = pd.read_csv('student.csv')
df

# DataFrame 확인
```

자주 사용하는 DataFrame 기능

1 전체 데이터 확인

```
df.head()
# 앞의 5개 행
```

결과 예시

	이름	국어	수학	영어	학년	반
0	김민수	85	90	88	2	1
1	이지현	92	88	90	2	1
...						

2 기본 정보 확인

```
df.info()
# 데이터 타입, 결측값 등
```

결과 예시

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5 entries, 0 to 4
Data columns (total 5 columns):
 # Column Non-Null Count Dtype
---
0 이름 5 non-null object
1 국어 5 non-null int64
...
```

3 통계 요약 확인

```
df.describe()
# 수치형 데이터 요약 통계
```

결과 예시

	국어	수학	영어	학년	반
count	5.00	5.00	5.00	5.00	5.00
mean	86.60	87.00	88.80	2.20	1.40
min	78.00	80.00	85.00	2.00	1.00
max	92.00	92.00	93.00	3.00	2.00

4 특정 열(컬럼) 선택

```
df['수학']
# 수학 점수만
```

결과 예시

0	90
1	88
2	85
3	92
4	80

Name: 수학, dtype: int64

5 여러 열 선택

```
df[['국어', '영어']]
# 국어, 영어 열만
```

결과 예시

	국어	영어
0	85	88
1	92	90
2	78	88
3	88	93
4	90	85

6 조건으로 필터링

```
df[df['수학'] >= 90]
# 수학 점수가 90점 이상인 학생
```

결과 예시

	이름	국어	수학	영어	학년	반
0	김민수	85	90	88	2	1
1	최수빈	88	92	93	2	1

7 정렬하기

```
df.sort_values('총점', ascending=False)
# 총점 기준 내림차순 정렬
```

결과 예시

	이름	국어	수학	영어	총점
3	최수빈	88	92	93	273
1	이지현	92	88	90	270
4	정우진	90	80	85	255
0	김민수	85	90	88	263
2	박준호	78	85	88	251

8 새로운 열 만들기

```
df['총점'] = df[['국어', '수학', '영어']].sum(axis=1)
# 국어 + 수학 + 영어
```

결과 예시

	이름	국어	수학	영어	총점
0	김민수	85	90	88	263
1	이지현	92	88	90	270
2	박준호	78	85	88	251
3	최수빈	88	92	93	273
4	정우진	90	80	85	255

9 특정 값으로 찾기

```
df[df['이름'] == '이지현']
# 이름이 '이지현'인 학생
```

결과 예시

	이름	국어	수학	영어	학년	반
1	이지현	92	88	90	2	1

10 결측값 확인 및 처리

```
df.isnull().sum()
# 열별 결측값 개수
```

결과 예시

이름	0
국어	0
수학	0
영어	0
학년	0
반	0
dtype:	int64



한 줄 요약

DataFrame은 데이터를 '쉽게 보고, 선택하고, 정리하고, 변환'할 수 있게 해주는 강력한 도구입니다.



Tip

여러 기능을 조합하면 복잡한 분석도 간단한 코드로 해결할 수 있습니다! 실습을 통해 직접 많이 사용해 보세요.

데이터 분석의 진짜 목적은 무엇일까?

데이터 분석은 단순히 데이터를 보는 것이 아니라, 더 나은 **의사결정**을 하기 위한 과정입니다.



핵심 메시지

데이터 분석의 궁극적인 목표는 **인사이트**를 발견하고, 더 나은 **의사결정**을 내리는 것입니다.

데이터 분석이 만들어내는 가치



우리 주변의 데이터 분석 예시

예시 1. 학생 성적 데이터



데이터를 분석하여 학생의 문제점을 파악하고, 더 나은 학습 결정을 내릴 수 있습니다.

예시 2. 카페 판매 데이터



데이터를 분석하여 고객의 선호를 파악하고, 매출을 높이는 전략을 세울 수 있습니다.



데이터 분석의 목적은 데이터를 많이 보는 것이 아닙니다.
데이터를 통해 의미를 찾고, 그 의미를 바탕으로 더 나은 결정을 내리는 것입니다.

데이터 분석가는 어떤 순서로 데이터를 다룰까?

데이터 분석은 정해진 순서대로
단계적으로 진행합니다.

이 순서를 이해하면
더 효과적으로 데이터를 다룰 수 있습니다.



핵심 포인트

- 모든 분석은 문제 정의에서 시작
- 데이터 품질이 분석 결과를 결정
- 분석 목적에 맞는 적절한 방법 선택
- 결과를 해석하고 의사결정에 활용

데이터 분석의 전체 과정



기억하세요!

좋은 분석은 좋은 질문에서 시작하고, 체계적인 과정을 통해 더 나은 답을 찾아냅니다.
순서를 이해하고 단계별로 접근하면 누구나 데이터 분석을 잘할 수 있습니다!

지금까지 배운 내용 정리

이번 강의에서는 데이터가 저장되는
방식부터 DataFrame을 활용하기까지의
전체 흐름을 살펴보았습니다.

오늘 배운 핵심 개념



Excel: 데이터를 표 형태로 관리하는 도구



CSV: 데이터를 저장하고 공유하는
가장 널리 사용되는 형식



Pandas: Python에서 데이터를 다루는
핵심 라이브러리



DataFrame: 표 형태의 데이터를 담는
Pandas의 핵심 자료구조

데이터가 분석되기까지의 전체 흐름



현실의 데이터

우리 주변에서 발생하는 다양한 데이터
(예: 학생 성적, 매출, 설문 조사 등)



Excel 파일

데이터를 표 형태로 입력하고 관리



CSV 파일

데이터를 저장하고 다른 곳과 쉽게 공유



Python

데이터 분석을 위한 프로그래밍 언어



Pandas

Python에서 데이터를 읽고 다루는 라이브러리



DataFrame

표 형태의 데이터를 저장하고 조작하는 구조



데이터 분석

조회, 정리, 통계, 시각화 등 다양한 분석 수행

반드시 기억할 것!



CSV는 가장 널리 사용하는
데이터 저장 형식입니다.



Pandas는 Python에서
데이터를 다루는 핵심 도구입니다.



DataFrame은 표 형태의
데이터를 담는 구조입니다.



대부분의 데이터 분석은
DataFrame을 기반으로 이루어집니다.



데이터 분석의 시작점은
DataFrame입니다.



핵심 정리

우리는 Excel → CSV → Python → Pandas → DataFrame의 흐름을 이해했습니다.
이 흐름을 이해해야 데이터 분석을 효과적으로 시작할 수 있습니다!

다음 시간 무엇을 배울까요?

이번 시간에는 Pandas와 DataFrame의 기초를 다졌습니다.

다음 시간에는 DataFrame을 더욱 효과적으로 다루기 위한 데이터 선택과 필터링 방법을 자세히 배워보겠습니다!



다음 시간의 목표

- ✓ 데이터 선택의 다양한 방법을 이해한다.
- ✓ 조건에 맞는 데이터를 필터링할 수 있다.
- ✓ 여러 조건을 조합하여 원하는 데이터를 추출할 수 있다.



기대 효과

데이터 선택과 필터링은 실제 분석에서 가장 많이 사용하는 핵심 기능입니다.
다음 시간 내용을 잘 익히면 원하는 데이터를 빠르고 정확하게 추출할 수 있게 됩니다!



01

데이터 선택하기

- 열(컬럼) 선택하기
- 행(인덱스) 선택하기
- 여러 열/행 선택하기

예시

특정 열만 보고 싶을 때,
특정 학생의 데이터만 보고 싶을 때

02

데이터 필터링하기



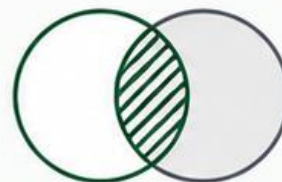
- 조건을 이용한 필터링
- 비교 연산자 활용
- 문자열 조건 필터링

예시

수학 점수가 80점 이상인 학생,
이름에 '김'이 포함된 학생

03

여러 조건으로 데이터 추출하기



- AND 조건 (모두 만족)
- OR 조건 (하나 이상 만족)
- NOT 조건 (조건 반대)

예시

수학 80점 이상 AND 국어 70점 이상
또는 과학 점수가 90점 이상인 학생

오늘 강의는 여기까지!

수고하셨습니다!

다음 시간에는 더 유용하고 재미있는
데이터 분석 방법을 배워봅시다.



이번 시간에 배운 내용

01



Excel 파일

데이터를 표 형태로
입력하고 관리

02



CSV 파일

데이터를 저장하고
공유하는 표준 형식

03



Python

데이터 분석을 위한
프로그래밍 언어

04



Pandas

Python에서 데이터를
다루는 핵심 라이브러리

05



DataFrame

표 형태의 데이터를 담는
Pandas의 핵심 구조

06



데이터 분석의 시작

DataFrame을 기반으로
다양한 분석 수행

다음 시간 예고



3강. 데이터 선택하기

데이터 분석에서 가장 기본이 되는
데이터 선택 방법을 배웁니다.

- ✓ 열(컬럼) 선택하기
- ✓ 행(인덱스) 선택하기
- ✓ 여러 열/행 선택하기
- ✓ 선택 결과 다양하게 활용하기



데이터 선택은 **분석의 첫걸음!**
다음 시간에 함께 배워봐요.



오늘의 한 줄 정리

데이터는 다양한 곳에 존재하며, **Pandas의 DataFrame**을 통해 우리는
데이터를 쉽게 이해하고 분석할 수 있습니다.